

Achievements Platform Reference

Date: 2026-07-27

Status: Living document — the operating manual for achievements

Audience: Engineers working on the API, the SDK, the Creator Portal, the mobile app, or a manufacturer integration

Scope note. This document describes the **v2 achievements platform**, which is the only achievements platform. The v1 API and the SDK's former v1-compatibility layer are retired; backward compatibility with v1 is no longer a requirement, and v1 endpoints and behaviours are deliberately not documented here except where a migration path still exists.

Every behavioural claim below was verified against source on 2026-07-27 and cites the file it came from. Where the original specification calls for something that is **not built yet**, it is described in full in [§14 Specified but not yet implemented](#) rather than being silently omitted — the taxonomy in the v3.0 specification is the target, and everything in it should eventually exist in v2.

Primary sources: [achievements_v3.0.pdf](#) (platform philosophy, 2024-12-14) ·

[PRD - Achievements UI and SDK.pdf](#) (2024-12-06) · [PRD - Achievements v2 and Creator Portal.pdf](#) (2026-02-09) · [../plans/jsa-achievement-v2-sdk-alignment-plan.md](#) · [docs/sdk/sdk-017 - sdk-025](#) · the [api](#) and [scorbit_sdk](#) repositories.

Table of contents

1. [What achievements are](#)
2. [Design philosophy](#)
3. [Platform architecture](#)
4. [Anatomy of an achievement](#)
5. [The rule engine](#)
6. [Where achievements are calculated](#)
7. [End-to-end lifecycles](#)
8. [API endpoints](#)
9. [Real-time channel](#)
10. [The SDK surface](#)
11. [Creating achievements](#)
12. [Data model](#)
13. [Operations](#)
14. [Specified but not yet implemented](#)

- 15. [In development](#)
- 16. [Known defects](#)
- 17. [Implementation status at a glance](#)
- 18. [Glossary](#)

1. What achievements are

An achievement is a badge attached to a **user profile** as a result of playing pinball. Players earn them, display them, and compete over them. Some are held by anyone who qualifies; some can be held by exactly one person at a time and can be taken away.

Achievements are the platform's main long-term engagement mechanic. The pitch is that a player can earn badges across fifty-plus years of pinball titles — a machine from 1995 and a machine from 2025 both contribute to one profile. Success is measured in engagement: repeat visits, app usage, and partner satisfaction.

Three groups create and consume them:

Audience	What they do
Players	Earn achievements, view them in the app and on machine displays, compete for trophies
Creators (manufacturers, designers, community)	Author achievements in the Creator Portal
Game integrators	Call the SDK from game code so the machine can evaluate and display achievements live

The single most important architectural sentence in the specification:

Achievements and their associated logic are **stored** in the Scorbit platform. In most cases, they are **processed** by the game or Scorbitron hardware.

That split — central storage, distributed evaluation — drives everything in §6.

2. Design philosophy

These are intent rules from the v3.0 specification. They are guidance for whoever designs achievement content, and they explain why the engine is shaped the way it is.

Achievements motivate play; they are not game mechanics. To motivate behaviour *during* a game, game designers should use **modes**, not achievements. Achievements are mostly valuable post-game, or at least post-ball, as a player evaluates how they did. An achievement that changes in-game decision-making is the exception, not the rule.

Most achievements are post-game. A player does not need to know the instant they qualified. The result can be computed after the session log is uploaded. This is the default and it scales.

Real-time achievements exist for exactly two reasons. Per the specification, an achievement should only be classified real-time if it needs to:

1. be displayed on the game's display during play, or
2. affect gameplay directly through game-code integration.

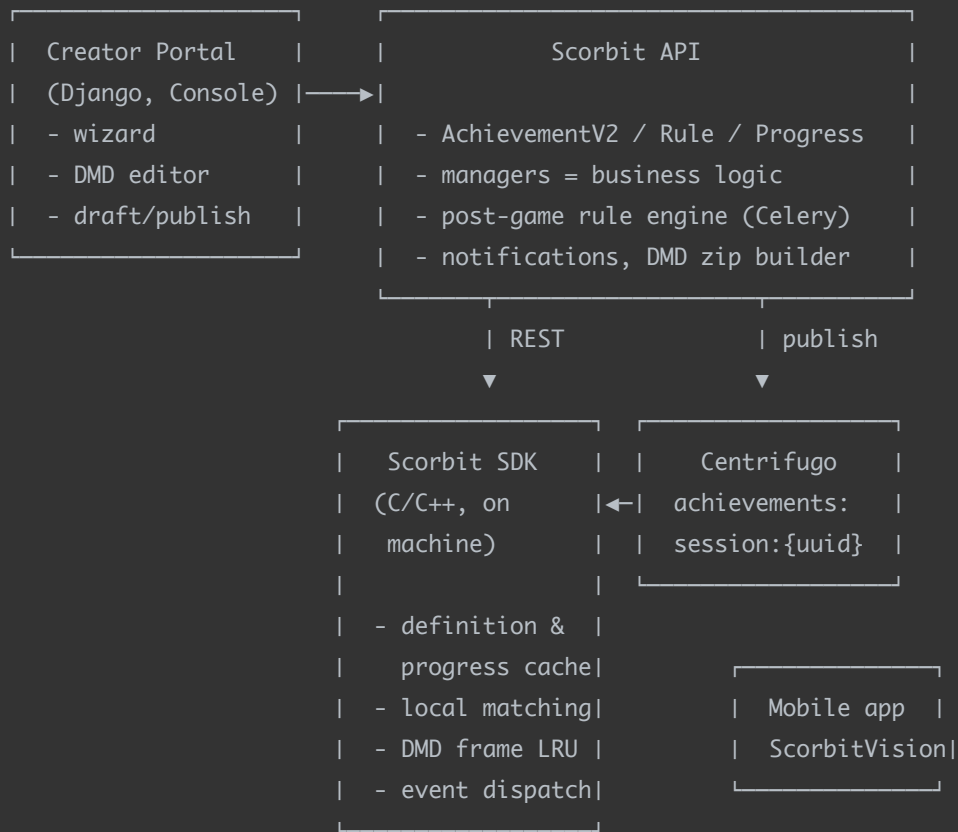
If neither is true, it belongs in post-game processing. This constraint exists because real-time evaluation must run locally on the machine, which costs device resources and integration effort.

Achievements are built on game state. For each game there are modes — `multiball` , `wizard` , and so on. A mode is a boolean, named by a string. Modern games know their own modes; retro games on Scorbitron hardware have modes inferred by the Scorbitron's client software from extracted display data. Achievements apply boolean algebra over those states.

Mystery is a feature. Achievements are delivered as quests. The player often does not know what it takes to earn one until they do, and then they socialise it. This is why *obscured* and *invisible* achievements exist (§4.4).

3. Platform architecture

3.1 Components



| callbacks



```
| Game code |  
| (manufacturer |  
| or Scorbitron) |
```

Component	Repo / path	Role
API backend	api	System of record. Stores definitions, rules, user progress. Runs post-game evaluation. Publishes real-time events.
Creator Portal	api — core/pages/creator.py , public/js/creator/	Web UI where achievements are authored, edited, tested, published, archived
SDK	scorbit_sdk — source/achievement_manager.* , include/scorbit_sdk/achievements*.h	On-machine cache + local rule matching + event delivery
Centrifugo	integrations/centrifugo/achievement.py	WebSocket transport for live unlock / lock / progress events
Game code	Manufacturer or Scorbitron	Reports modes and scores, calls unlock, renders DMD frames

3.2 Transport split

REST carries durable state; Centrifugo carries live notification.

Data	Transport	Why
Achievement definitions	REST	Bulk, cacheable, changes rarely
User progress snapshot	REST	Fetches at player claim
Unlock / lock requests	REST	Must be reliable and idempotent
DMD frames	REST (zip)	Large binary, cached on device
Unlock / lock / progress events	Centrifugo	Low latency, fire-and-forget acceptable

3.3 Where state lives

State	Home	Notes
Achievement definitions	achievement_v2 table	Authoritative
Rules	rule table, FK to achievement	Authoritative
Per-user unlock status	achievement_v2_userachievement	Authoritative
Per-user/per-game counters	progress table, metrics JSONField	Feeds PROGRESS/MODE/SCORE server rules
Definition + progress cache	SDK memory	Rebuilt each boot; not persisted
DMD frames	SDK LRU cache, 32 frames	source/achievement_manager.h

4. Anatomy of an achievement

4.1 Identity — the key

Every achievement has an **immutable, unique key**. Game code references achievements by key forever; the key never changes once created. Format and rules are specified in [achievement-key-format.md](#):

```
{scope}-{context}-{identifier}
```

game-cv-boom-balloon · venue-flipnout-regular-visitor · event-tpf2025-tournament-winner · global-first-game

Max 100 characters; lowercase, digits and hyphens only; auto-generated from the name but editable before first save.

4.2 Scope

Scope answers "what does this achievement attach to?" Four values (`core/models/achievement.py` , `SCOPE_CHOICES`):

Scope	Meaning	Status
game	Tied to a game title , not a machine instance. The common case.	Implemented
venue	Earned only while playing at a specific venue	Partial — see §14.6
event	Earned only during a specific event/tournament	Partial — see §14.6
global	Applies across titles; may not relate to a specific game state	Implemented (post-game only)

Note the deliberate title-vs-variant decision: a `game`-scoped achievement applies to *Theatre of Magic*, not to one particular Theatre of Magic cabinet. v1 scoped achievements to machines; v2 scopes to games because that is how players and manufacturers think. `SPECIFIC` remains as an alias for `GAME`.

4.3 Permanence

Type	Behaviour
Permanent	Once earned for a game, stays on the profile indefinitely. The most common type.
Trophy	Held by exactly one player at a time — the Foursquare-mayor model. Winning it takes it from the previous holder, who is notified. Cannot be permanent.

Because achievements are *unlocked*, "locking" is the reverse — the act of taking one away. That is what the lock endpoint and the `AchievementLocked` event mean.

4.4 Visibility

Three independent ideas, often confused:

Property	Effect
<code>visible</code>	If false, the achievement is an ingredient — it never appears in player-facing lists. Used for nesting chains so intermediate steps don't clutter the UI.
<code>obscure</code>	The achievement is listed, but its name/description/image are replaced by a placeholder (<code>obscure_image</code>) until earned. Used to create mystery, typically on the top tier of a group.
<code>status</code>	<code>draft</code> / <code>published</code> / <code>archived</code> — an authoring workflow state, not a player-facing one. Only <code>published</code> reaches players.

Rendering contract for clients: if `obscure` and not yet unlocked, show `obscure_image_url` with a masked name; if `!visible` and not unlocked, show nothing at all.

4.5 Session scope

`is_single_session` decides whether progress resets:

Value	SDK enum	Meaning
<code>true</code>	<code>AchievementInputTime::Limited</code>	Progress counts within one game session only
<code>false</code>	<code>AchievementInputTime::Unlimited</code>	Progress accumulates across sessions for life

The SDK does **not** currently enforce the reset for `Limited`. See §16.

4.6 Groups and levels

A group associates related achievements — classically a tier ladder: 100 spins, 1,000 spins, 10,000 spins. Members share a `group_id` and are ordered by `level`.

Levels use **sparse integers**, default gap 1000, so a new tier can be inserted between two existing ones without renumbering. The API also returns `display_position`, a dense 1,2,3... sequence for UI ordering.

```
{ "key": "game-cv-boom", "level": 1000, "display_position": 1 }
{ "key": "game-cv-blast", "level": 2000, "display_position": 2 }
{ "key": "game-cv-new", "level": 2500, "display_position": 3 }
```

Groups are for display and progression legibility — they are not aliases and carry no evaluation semantics. A level-3 badge is typically the level-2 art with a level overlay.

4.7 Nesting and sub-achievements

Any achievement can require other achievements first, via an `ACHIEVEMENT` rule pointing at a subachievement . This produces scavenger-hunt style quests, and combined with `visible: false` produces long invisible chains that surface only at the final visible step.

The canonical example from the specification, *Star Trek Multiballer* (10 multiballs across any number of Star Trek games):

```
Star Trek Multiball 1 = achieve multiball (invisible)
Star Trek Multiball 2 = Star Trek Multiball 1 + achieve multiball (invisible)
...
Star Trek Multiballer = Star Trek Multiball 9 + achieve multiball (visible)
```

4.8 Counters

An achievement is either **boolean** (one completion) or a **counter** (`count` completions). Counter achievements report incremental progress and drive progress bars.

4.9 Comparison operators

Every rule carries its own comparison operator, applied to whatever that rule measures — score, counter, duration, sub-achievement presence. Three choices, from `Rule.COMPARISON_CHOICES` (`core/models/rule.py:41`):

Stored	Constant	Meaning
>	GT	Greater than
<	LT	Less than
=	EQ	Equal

Default is `>` .

Operators are per-rule, not per-achievement. This is a genuine v2 improvement: the v1 `Achievement` model carried a single flat `comparison` field (`core/models/achievement.py:164`), so one achievement had one operator. In v2 the field lives on `Rule` , so a multi-rule achievement can mix operators — "score > 1,000,000,000 **and** duration < 60,000 ms" is expressible as two rules.

The Creator Portal exposes the operator per rule. The form posts a parallel `rule_comparison[]` array alongside `rule_type[]`, `rule_target[]` and `rule_reference[]` (`core/pages/creator.py`:593 , :1022 , :1346), and the Vue form renders a `<select>` bound to `rule.comparison` (`templates/creator/CreatorAchievementForm.vue`:703 , :796 , :833). Labels are contextual rather than raw symbols — a counter rule offers **At least** (`>`) / **Exactly** (`=`), while score and threshold rules offer the full **Greater than** / **Equal to** / **Less than** set. Some rule types pin the operator with a hidden `=` input where a comparison would be meaningless. Everything round-trips: the wizard rehydrates `rule_comparison[]` from stored rules (`core/pages/creator.py`:1242) and defaults to `>` when a value is absent.

See §5.3 for a caveat: on rule types the server cannot measure, `<` and `= 0` defeat the fail-safe that otherwise prevents the server granting an achievement it has no way to evaluate.

4.10 Artwork

Asset	Purpose
<code>icon / image_url</code>	High-resolution badge, shown in the app and on LCD machines. May be a GIF or Lottie animation.
<code>obscure_image</code>	Placeholder silhouette shown before an obscured achievement is earned
<code>frame</code> (DMD)	Pixel art at the exact resolution and palette of a specific DMD machine, overlaid on the game display at unlock. Supports transparency.

DMD frames matter for games built roughly 1990–2015. They are packaged per game into a versioned ZIP; a machine re-downloads only when the version changes since its last check-in, so the frame is already local when the moment comes to display it.

5. The rule engine

5.1 Rules are boolean algebra over game state

An achievement holds one or more **rules**. All evaluable rules must be satisfied simultaneously — the combinator is **AND**. An achievement with a `MODE` rule and a `SCORE` rule requires the mode *and* the score.

There is no `OR` combinator. Alternatives are modelled as two achievements in a group, or as a chain of sub-achievements.

5.2 The ten rule types

Defined in `core/models/rule.py`. This is the complete set.

Type	Meaning	Evaluated by	Status
MODE	A specific mode was completed	SDK, in-session	Implemented
MODE_START	A specific mode was started	SDK, in-session	Implemented
MODE_STACK	Multiple modes active simultaneously	SDK, in-session	Implemented
SCORE	Score reached a threshold	SDK, in-session	Implemented
PROGRESS	Counter metric on the user/game Progress record	Server, post-game	Implemented
ACHIEVEMENT	Another achievement must be earned first	Server, post-game	Implemented
GAME_CODE	Custom logic private to the game	Game code	By design — see §14.1
TIMER	Elapsed time from a mode's start	—	Not implemented — §14.2
EVENT	A specific event the Scorbitron matches	—	Not implemented — §14.3
ATTEMPT	Whether this was the first try during the game	—	Not implemented — §14.4

Rule fields: `type` , `comparison` , `target` , `reference` (mode name or metric key), `subachievement` (FK, required for `ACHIEVEMENT`), plus `completed` / `completed_at` .

5.3 Server-side evaluation semantics

`Rule.get_current_value(user)` (`core/models/rule.py:81`) resolves a rule to a number:

- `PROGRESS` , `MODE` , `SCORE` → `Progress.metrics[rule.reference]` for that user and game, or `0` if no Progress record exists.
- `ACHIEVEMENT` → `1` if the `subachievement` is achieved by the user, else `0` . Raises `ImproperlyConfigured` if no subachievement is set.
- `GAME_CODE` , `TIMER` , `EVENT` , `ATTEMPT` , `MODE_STACK` , `MODE_START` → `0` , because these are not server-measurable.

`Rule.is_satisfied(user)` then applies the comparison.

Important consequence. Returning `0` for SDK-evaluated types is a deliberate fail-safe: with the default `>` comparison and a target of 1 or more, the rule is never satisfied server-side, so the server cannot accidentally grant an achievement it has no way to measure. **This safety depends on the comparison.** A `<` comparison (`0 < target → true`) or an `=` comparison against target `0` will be satisfied trivially. Authors should not use `<` or `= 0` on SDK-evaluated rule types.

Sub-achievement chains rely on this arithmetic: an `ACHIEVEMENT` rule with `> 0` is satisfied when the prerequisite is earned.

5.4 Worked examples

From the v3.0 specification, expressed as v2 objects.

Bride of Pinbot Billionaires Club — global, permanent, score over one billion:

```
scope: global · is_trophy: false
rules: [ { type: SCORE, comparison: ">", target: 1000000000, reference: "score" } ]
```

Theatre of Magic Grand Finale — game-scoped, permanent, mode completed on any ToM machine:

```
scope: game · game: Theatre of Magic
rules: [ { type: MODE, comparison: ">", target: 0, reference: "Grand Finale" } ]
```

Keith Elwin Lover — global, permanent, played three specific titles. Modelled as a chain, because rules AND within one game context:

```
global-played-jurassic-park    (invisible)
global-played-iron-maiden     (invisible)
global-played-archer          (invisible)
global-keith-elwin-lover      rules: three ACHIEVEMENT rules, "> 0" each
```

Keith Elwin Free Gold Watch Champion — venue-scoped trophy requiring top score on two machines at one venue. Expressible today only in part; the relative "highest score" predicate is [§14.7](#).

6. Where achievements are calculated

This is the question that most often gets answered wrongly. There are **two engines**, and which one runs an achievement is determined by its rule types and scope — not by a flag on the record.

6.1 The two engines

	In-session engine	Post-game engine
Runs on	The machine — SDK inside game code or Scorbitron	API server, Celery worker
Implementation	source/achievement_manager.cpp	core/tasks/achievement.py::evaluate_post_game_achievements
When	The moment game code asks, during play	After the session completes and uploads
Latency	Immediate	Seconds to minutes; deliberately unbounded
Input	Cached definitions + cached user progress + live modes/score	Session record + Progress.metrics + existing unlocks
Handles	MODE , MODE_START , MODE_STACK , SCORE	ACHIEVEMENT chains, PROGRESS counters, all global scope
Authority	Predictive only	Authoritative

6.2 Routing table

Rule type in the achievement	Evaluated where
Only MODE / MODE_START / MODE_STACK / SCORE	In-session; unlock posted to API by the game
Any ACHIEVEMENT rule	Post-game, server
Any PROGRESS rule	Post-game, server
Scope global	Post-game, server, regardless of rule types
GAME_CODE	Game code decides and calls unlock explicitly

The SDK actively **skips** ACHIEVEMENT and PROGRESS rules during local matching, and the server returns 0 for the SDK-owned types. The two engines are complementary by construction, not overlapping.

6.3 The server is always the authority

Local matching exists for **UI anticipation only**. The machine may light up a "you're close" indicator, but the truth is what the server says. The SDK documentation is emphatic about this and it is the correct contract: never show "unlocked" off the back of a local match — wait for the AchievementUnlocked event.

Reasons this matters: the machine cannot see cross-session counters, cannot see other players' trophy claims, and cannot arbitrate a trophy transfer.

6.4 Post-game trigger path

```

session completes
  → session_completed signal (core/signals.py:518)
  → guard: session.successfully_completed must be true
  → transaction.on_commit(...)
  → evaluate_post_game_achievements.delay(session.id) [Celery, queue="default"]
    → for each claimed user on the session:
      → published achievements for this game having ACHIEVEMENT rules → check_completion
      → published achievements with scope=global → check_completion
        → UserAchievementV2.objects.check_and_unlock(user, achievement, session)
          → skip if already achieved
          → skip if the achievement has no rules
          → all(rule.is_satisfied(user)) → unlock + publish + notify

```

Note the two guards worth remembering operationally: an achievement with **no rules can never be auto-unlocked**, and an already-achieved achievement short-circuits immediately.

7. End-to-end lifecycles

7.1 Machine boot

1. SDK authenticates and establishes its session.

2. SDK fetches all published achievement definitions for the machine's game — `GET v2/achievements/scorbitron/` .
3. Definitions, including the nested `rules` array, are cached in memory.
4. SDK compares the DMD frame ZIP version against its local copy and downloads only if changed.

7.2 Player claims a slot

1. Player taps NFC or claims in the app.
2. SDK fetches that player's progress — `GET v2/achievements/scorbitron/progress/` .
3. Progress merges with cached definitions to yield earned/unearned/in-progress per achievement.
4. The API subscribes the session to `achievements:session:{session_uuid}` on Centrifugo (`source/net.cpp:859` , at session creation).

7.3 In-session unlock

1. Game code reports state: `sb_add_mode / sb_remove_mode / sb_set_score` , then `sb_commit` .
2. Game code asks the SDK what might qualify — `sb_check_mode_achievements` , `sb_check_score_achievements` , or the combined mode+score variant.
The SDK does not evaluate automatically on state change; the game must ask.
3. For a match, game code calls `sb_unlock_achievement(handle, user_id, key, count, cb)` .
4. Server validates, records, publishes to the session channel, and queues notifications.
5. SDK receives `AchievementUnlocked` , dispatches it to the game's event callback.
6. Game displays the cached DMD frame — immediately, or deferred to end of ball.

7.4 Post-game

1. Session ends; the log is uploaded.
2. `session_completed` fires post-game evaluation (§6.4).
3. Newly satisfied chains and global achievements unlock.
4. Profile updates; the player and their followers are notified.

7.5 Trophy transfer

1. A player meets a trophy condition.
2. Client calls `POST v2/achievements/lock/` to move the trophy.
3. Previous holder loses it — `AchievementLocked` published, holder notified.
4. New holder gains it — `AchievementUnlocked` published.

Trophy transfer is currently **client-initiated**. The specification's post-game examples (e.g. "longest game of Jurassic Park at Free Gold Watch") imply the server should recompute holders after each session. See §14.8.

8. API endpoints

All under `/api/v2/`. Routes registered in `api/v2/urls.py`.

8.1 Device / SDK endpoints

These four are what the SDK actually calls (`source/identifiers.h:38-41`).

Method	Path	Purpose
GET	<code>v2/achievements/scorbitron/</code>	All published achievements for the device's game, with nested rules and frame URLs, in one response
GET	<code>v2/achievements/scorbitron/progress/</code>	The claimed user's progress for those achievements
POST	<code>v2/achievements/unlock/</code>	Unlock or update counts, keyed by achievement key
POST	<code>v2/achievements/lock/</code>	Revoke a trophy from its holder

Unlock request (`api/v2/serializers/achievement.py`):

```
{
  "achievements": [ { "key": "game-cv-boom-balloon", "count": 3 } ],
  "user_id": 12345,
  "session_uuid": "...",           // optional; enables real-time publish
  "score_id": 678                  // optional; retained for non-SDK clients
}
```

The response reports per-achievement outcome including whether each was **newly** unlocked, so clients can suppress duplicate celebration.

`session_uuid` is what makes an unlock visible in real time. Without it the unlock is recorded but no Centrifugo event is published (`core/managers/achievement_v2.py:416`).

8.2 Creator Portal and management endpoints

Method	Path	Purpose
GET/POST/PATCH/DELETE	v2/achievements/	Achievement CRUD
GET/POST/PATCH/DELETE	v2/achievements/{id}/rules/	Nested rule management
POST	v2/achievements/{id}/...	Publish, archive, and related state actions
POST	v2/achievements/import-csv/	Bulk import (transitional — see §13.4)
GET	v2/games/{game}/achievements/	Achievements for a game title
GET	v2/games/{game}/achievements/frames-zip/	Versioned DMD frame bundle
GET	v2/users/{user}/achievements/	A user's achievements
GET	v2/users/{user}/progress/	A user's per-game progress metrics

Achievement records use integer primary keys, not UUIDs — a documented legacy exception, so `ViewSet` set `lookup_value_converter = "int"`.

Creator write access is gated by `IsAchievementCreator` (`api/v2/permissions.py:169`).

9. Real-time channel

9.1 Channel

```
achievements:session:{session_uuid}
```

One channel per game session (`integrations/centrifugo/achievement.py:49`). The SDK subscribes when the session is created and unsubscribes when it ends. Publishing happens from the achievement managers when a `session` is in scope.

9.2 Event types

Type	Payload highlights	Meaning
achievement_unlocked	key, name, user_id, username, icon_url, is_trophy	A player earned it
achievement_locked	key, name, user_id, username, icon_url	A trophy was revoked
achievement_progress	key, name, user_id, username, icon_url, current_value, target_value	Counter moved

The SDK converts each into a typed event (`AchievementUnlockedEvent` etc.) at `EventPriority::High` and delivers it through the event manager to the game's callback (`source/net.cpp:1990`, `handleAchievementPublication`).

9.3 Client guidance

- Callbacks must return fast. Queue artwork loading and animation; never block the event loop.

- After a reconnect you may receive events for unlocks that happened while disconnected — deduplicate on `key + user_id`.
- Both a progress event at 100% and an unlock event will arrive for the same achievement. Show one celebration, not two.

Only the per-session channel exists. The specification also described per-game, per-user, and per-machine channels — see §14.9.

10. The SDK surface

10.1 Language support — read this first

Language	Achievement support	Notes
C	Complete — 18 functions	The real, working surface
C++	Types only	<code>achievements.h</code> defines structs, enums and callback aliases. The public <code>GameState</code> class in <code>game_state.h</code> has no achievement methods .
Python	None	No achievement bindings exist in <code>wrappers/python/</code>

This is a live discrepancy with the SDK documentation, which shows C++ and Python examples throughout — including `gameState.fetchAchievements(...)` in the `achievements.h` header comment itself, which does not compile. **Integrate against the C API.** Tracked in §16.

10.2 C API

Declared in `include/scorbit_sdk/achievements_c.h`.

Network operations

Function	Purpose
<code>sb_fetch_achievements</code>	Fetch and cache definitions for the game
<code>sb_fetch_achievement_progress</code>	Fetch and cache a user's progress
<code>sb_unlock_achievement</code>	Unlock / update a count on the server
<code>sb_lock_achievement</code>	Revoke a trophy

Cache access

Function	Purpose
<code>sb_has_achievements</code>	Whether definitions are cached
<code>sb_get_cached_achievements_count</code>	Number cached
<code>sb_get_cached_achievement_at</code>	Fetch by index
<code>sb_get_cached_achievement</code>	Fetch by key
<code>sb_get_cached_progress</code>	A user's progress for one achievement
<code>sb_achievement_get_rules_count</code>	Rule count for an achievement
<code>sb_achievement_get_rule_at</code>	Fetch one rule

Local matching

Function	Purpose
<code>sb_check_mode_achievements</code>	Keys matching a mode start/complete/stack
<code>sb_check_score_achievements</code>	Keys matching a score threshold
<code>sb_increment_achievement_progress</code>	Bump a local counter; returns true on newly-unlocked
<code>sb_set_achievement_triggered_callback</code>	Simplified local-trigger callback

DMD frames

Function	Purpose
<code>sb_download_achievement_frames</code>	Populate the frame cache
<code>sb_has_dmd_frame</code> / <code>sb_get_dmd_frame</code>	Query the 32-entry LRU cache

Events arrive through the general event callback set on `Config` **before** the `GameState` is created, then decoded with `sb_event_achievement_unlocked` / `_locked` / `_progress` (`event_helpers.c.h`).

10.3 Local matching semantics

`AchievementManager` (`source/achievement_manager.cpp`) evaluates cached definitions against the state the caller passes in:

- All evaluable rules must be satisfied — same AND semantics as the server.
- `ACHIEVEMENT` and `PROGRESS` rules are skipped, not failed.
- `GAME_CODE`, `TIMER`, `EVENT`, `ATTEMPT` are skipped — game code owns them.
- Already-unlocked achievements are skipped when `inputTime == Unlimited`.
- An achievement whose rules are *all* skipped never matches, because at least one evaluable rule is required.

The SDK never evaluates on its own. `setScore`, `addMode`, `removeMode` and `commit` contain no achievement logic — they only mutate state. Evaluation happens when game code calls a `check_*` function. The cadence of achievement checking is therefore entirely the integrator's choice.

10.4 Derived convenience fields

For single-rule achievements the SDK derives flat fields — `trigger`, `mode_name`, `mode_type`, `target_score` — from the first matching rule. For multi-rule achievements these are lossy; inspect the `rules` array instead.

11. Creating achievements

11.1 Access

Creators need `creator` group membership plus an explicit assignment: specific game titles, or a manufacturer group covering all of that manufacturer's titles. A creator sees and edits only their own scope. Implemented via `CreatorAccessMixin` (`core/pages/creator.py:181`) and `IsAchievementCreator`.

11.2 Authoring flow

Dashboard → list by game title, searchable by name, key, or manufacturer prefix, filterable by status → create or open an achievement → wizard or detail editor.

The wizard follows "when this and/or that is true, compared to a value, then achieved = true":

```
Name → Description → Template or blank
  → Scope (game / venue / event / global) → Permanence (permanent / trophy)
  → Group and level → Session scope (single-session / lifetime)
  → Trigger type → Comparison → Target value
  → Visibility (visible / invisible / obscured)
  → Badge upload → DMD frame → Notification preference
  → Save as Draft
```

An achievement cannot be orphaned — it must relate to a game, group of machines, venue, event, or another achievement.

11.3 Draft, publish, archive

Status	Meaning
draft	Default on save. Not visible to players. Creator can test against their own account.
published	Live for all players.
archived	Soft delete — hidden from view. Published achievements are archived, never unpublished.

Only `published` achievements are served to devices or evaluated post-game.

11.4 DMD frame editor

Modelled on Pinball Browser's "Scorbit Achievement Editor" (`public/js/creator/dmd-editor.js`):

- Pixel-level editing at the machine's actual display geometry
- Palette of 4, 8, or 16 shades depending on the machine's encoder
- Bitmap fonts derived from WPC and Stern machines for text
- Layers, so a badge graphic can be composited with text
- Transparency
- Upload an existing frame, or copy from another achievement

Saving merges visible layers, writes the frame, and triggers a rebuild of the per-game ZIP with an incremented version — which is what causes machines to re-download.

11.5 Testing

Creators can trigger an achievement against their own account, list the conditions an achievement requires, and preview app and DMD presentation before publishing.

12. Data model

`core/models/achievement.py` , `core/models/rule.py` , `core/models/progress.py` .

AchievementV2

Group	Fields
Identity	<code>id</code> (int pk), <code>key</code> (unique, immutable), <code>name</code> , <code>description</code>
Scope	<code>scope</code> , <code>game</code> FK, <code>venue</code> FK, <code>event</code> FK
Classification	<code>is_badge</code> , <code>is_trophy</code> , <code>is_single_session</code> , <code>is_active</code>
Visibility	<code>visible</code> , <code>obscure</code> , <code>obscure_image</code>
Grouping	<code>group_id</code> , <code>level</code> , <code>display_position</code> (computed)
Artwork	<code>icon</code> , <code>frame</code> , <code>frame_image</code> , <code>frame_version</code>
Workflow	<code>status</code> , <code>draft</code> , <code>creator</code> , <code>users</code> (M2M with <code>owner</code> / <code>viewer</code> roles)
Behaviour	<code>notify_when_achieved</code> , <code>ball_count</code> , <code>duration</code>
Audit	<code>created_at</code> , <code>updated_at</code>

Helper methods include `get_progress` , `check_completion` , `publish` , `archive` , `can_user_edit` , `get_next_level` , `get_insert_level` , `rebalance_group` .

Rule

FK to achievement (`related_name="rules"`), optional FK to `subachievement` , plus `type` , `comparison` , `reference` , `target` , `completed` , `completed_at` .

UserAchievementV2

Links user to achievement with `achieved` , achieved time, and current counter value. Managed through `UserAchievementV2Manager` — `unlock` , `lock` , `check_and_unlock` , `update_progress` .

Progress

One row per user per game, with a `metrics` `JSONField`. This is the substrate for server-side `PROGRESS` , `MODE` and `SCORE` rules: `rule.reference` is the metric key. `ProgressManager.update_metrics` (`core/managers/achievement_v2.py:640`) applies deltas.

13. Operations

13.1 Celery tasks

Task	Queue	Role
<code>evaluate_post_game_achievements</code>	<code>default</code>	Post-game rule engine
<code>send_achievement_v2_notifications</code>	<code>high</code>	Player and follower notifications
<code>generate_dmd_zip</code>	<code>high</code>	Rebuild one achievement's frame bundle
<code>generate_game_dmd_zip</code>	<code>high</code>	Rebuild a game's frame bundle

`evaluate_post_game_achievements` is Sentry-sampled at 0.01 (`core/settings.py:1473`).

13.2 Notifications

Unlock notifications reach the earning player and their followers. Templates live in `PushNotificationContent` (fixture `core/fixtures/notifications.jsonl`) using `%(name)s` -style formatting — a missing context key raises `KeyError` , so prefer separate slugs over blank values. `notify_when_achieved` controls whether followers are notified.

13.3 Frame versioning

Frames are bundled per game into a versioned ZIP. Devices check version at power-up and on config check, and download only on change. Editing any frame increments the version and rebuilds the bundle.

13.4 CSV import

CSV import survives as a transitional bulk-entry path (`v2/achievements/import-csv/`) for creators adding many achievements at once. It is deprecated in favour of the Creator Portal; the intended end state is a bulk-upload feature inside the Portal rather than a standalone workflow.

14. Specified but not yet implemented

Everything in this section is described in the v3.0 specification or the PRDs and is **intended to exist** in v2. It is documented here in full so the design intent is not lost, and so that whoever implements it does not have to re-derive it from PDFs.

Check §15 before picking anything up

from here. Several of these items are already designed and scheduled in the active Achievement Platform initiative — notably `TIMER/duration`, relative targets, and automatic trophy recomputation. Items carrying a **Scheduled** note below are covered there.

14.1 `GAME_CODE` rules

Intent. An escape hatch for logic only the game knows — "player completed the ramp combo without dropping a ball" — that the platform cannot express.

Status. Defined and deliberately unevaluated on both sides. The server returns `0`; the SDK skips it. Game code is expected to determine satisfaction itself and call `unlock` directly.

To do. Nothing structural. Worth documenting for integrators that an achievement with a `GAME_CODE` rule will never auto-unlock and *must* be explicitly unlocked by the game.

14.2 `TIMER` rules and `duration`

Intent. "The mode or target must be completed in less than or greater than a specified period of time," measured in seconds or milliseconds from a mode's start. This is what makes the PRD's flagship example — *Marathon Multiball*, "maintain multiball for 60 seconds" — expressible.

Status. The `TIMER` rule type exists and the `duration` field exists on the model and is editable in the Creator Portal (`core/pages/creator.py:442`), and was carried through the v1 migration. **Nothing evaluates either.** The server returns `0`; the SDK skips `TIMER`.

To do. Decide the owner. Timing is inherently in-session, so the SDK is the natural evaluator: it would need to record a start timestamp when a mode is added and compare elapsed time against `target` when the mode completes. Requires a mode-start timestamp map in `AchievementManager` and a new local-matching entry point.

Scheduled. Epic 1 approaches this from the server instead: `SessionModeInstance` stores one row per mode activation, making `duration` a direct query, and `mode:<name>:duration_ms` joins the metric vocabulary. That covers post-game duration achievements; live on-DMD duration display would still need the SDK-side work above.

14.3 `EVENT` rules

Intent. "Some specific event the Scorbitron matches to" — a discrete occurrence detected from extracted display data that is not a mode, e.g. a specific award or animation.

Status. Type defined; nothing evaluates it.

To do. Requires a vocabulary of event names shared between PatternsTrainer/Scorbitron extraction and the rule `reference` field, plus an SDK entry point analogous to `checkModeAchievements` .

14.4 ATTEMPT rules

Intent. "Mostly, was this the first try during the game" — qualify an achievement on a first-attempt basis so that grinding retries within one session does not count.

Status. Type defined; nothing evaluates it.

To do. Requires per-session attempt counting for the referenced mode or target, almost certainly in-session in the SDK.

14.5 Ball count

Intent. "A specific ball number can be specified if an achievement must be completed before it increments. For example, you must complete a multiball on ball 1, but before ball 2."

Status. `ball_count` is stored, editable in the Creator Portal, and migrated from v1 — but no rule type references it and no engine evaluates it. The SDK's parser maps the API's `ball_count` onto `Achievement.count` , the counter threshold, which is a different concept (see §16).

To do. Either introduce a `BALL` rule type, or define `ball_count` as a session-scoped qualifier that the SDK checks against current ball before matching. The current field placement implies the latter.

14.6 Venue and event scope enforcement

Intent. A venue achievement is earned only when the criteria are met *at that venue* — "play any Dutch Pinball game during Texas Pinball Festival 2025 and earn a TPF 2025 achievement." Event scope is the same idea bounded by time.

Status. Partial. `scope` , `venue FK` and `event FK` all exist, and delivery is filtered — the Scorbitron queryset restricts to achievements for the venue or with no venue (`core/managers/achievement_v2.py:99`). But `check_and_unlock` does **not** verify that the unlock occurred at the required venue or during the required event.

To do. Add venue/event verification inside the unlock path, using the session's `venuemachine` and timestamp. Until then, venue scoping is a delivery filter, not a guarantee — a client could unlock a venue achievement from the wrong venue.

14.7 Relative and comparative targets

Intent. The specification allows score ingredients that are *relative* rather than absolute: "player has highest batphone champion on Batman '66," or "longest game of Jurassic Park at Free Gold Watch." These are leaderboard predicates, not thresholds.

Status. Not implemented. `Rule.target` is a `PositiveIntegerField` compared with `>`, `<`, `=` against a scalar. There is no way to express "is the maximum among players."

To do. Needs a new rule type or a target mode that resolves against a leaderboard query at evaluation time. This is a prerequisite for most trophy achievements being automatic rather than client-asserted, and pairs with §14.8.

Scheduled. Epic 2.3 implements exactly this predicate for trophies — recomputing the top holder from `SessionModeInstance + Score` in one SQL statement per trophy. Note the scope: it delivers relative ranking for the **trophy holder** path, not a general relative comparison usable by any rule.

14.8 Automatic trophy recomputation

Intent. From the specification's Example 6: a player completes a game, the log is uploaded, post-game processing determines it is the longest game of that title at that venue, the achievement is removed from the previous holder (who is notified) and added to the current player (who is notified).

Status. The mechanism for transfer exists — `POST v2/achievements/lock/`, the `AchievementLocked` event, and notifications on both sides. What is missing is the server deciding *on its own* that a transfer should happen. Today a client must assert it.

To do. Post-game evaluation should recompute trophy holders for trophy achievements whose predicate is leaderboard-relative, then perform the lock/unlock pair. Depends on §14.7.

Scheduled. Epic 2.2–2.4 and 2.7 build precisely this: an `AchievementHolder` table, a `SELECT ... FOR UPDATE` swap transaction that revokes and re-grants atomically, `update_trophy_holder` scheduled off the post-session chain, debouncing so a busy venue does not melt the queue, and the `achievement-trophy-revoked` notification. See §15.3.

14.9 Additional real-time channels

Intent. Four channels were specified: per-session (Scorbitron), per-machine (DMD display triggers), per-user (personal notifications to the app), and per-game (spectator/leaderboard feed).

Status. Only `achievements:session:{uuid}` exists.

To do. `achievements:user:{id}`, `achievements:machine:{id}`, `achievements:game:{slug}`. Note that app notifications are currently served by push, so the per-user channel is an enhancement rather than a hole; the per-game spectator feed has no substitute today.

14.10 Ephemeral session achievement array

Intent. PRD requirement: when a user unlocks an achievement during a live session, add it to an array associated with that session and player in **Redis**, for score egress and for querying during the session. Explicitly ephemeral — not persisted — so the app and ScorbitVision can show what a player earned during *this* game for post-game analytics.

Status. Not implemented. No Redis usage exists in the achievement managers or the Centrifugo achievement integration. The PRD notes prior partial work in the score pipeline that was never finished.

To do. Session-scoped Redis structure written on unlock, exposed for score egress and readable during the session.

14.11 Explicit real-time vs post-game classification

Intent. The specification is emphatic: "during the creation of an achievement, you must decide what type of achievement it will be and how it will be managed by the platform." The v1 model had a processing field with `immediate` / `postgame` choices, and the Creator UI mock shows a "Process Real-Time / Post-Game" toggle.

Status. AchievementV2 has **no** processing field. Routing is implicit, derived from rule types and scope (§6.2).

To do. Decide deliberately whether implicit routing is the final answer. Implicit routing is arguably better — it cannot disagree with the rules — but it removes the author's ability to force an otherwise-post-game achievement to be evaluated live for display purposes, which is one of the two stated reasons real-time achievements exist at all.

14.12 Geo-fencing as an ingredient

Intent. "Global Achievements may use geo fencing as an ingredient, or other user accomplishments" — e.g. "first time entering all the bars in San Francisco and playing a game."

Status. Not implemented; no location predicate exists in the rule engine.

14.13 Animated DMD frames

Intent. Open question in the v2 PRD: should the frame editor support multi-frame sequences, or are static frames sufficient for launch?

Status. Static frames only. Badge artwork (`icon`) already supports GIF and Lottie; DMD frames do not.

14.14 Achievement templates

Intent. Pre-built templates for common patterns — "count during a game," "count lifetime," "achievement(s) achieved previously," "mode stacking," "score threshold" — that pre-fill the wizard.

Status. Template loading exists in the Creator Portal (`_load_template_data` , `core/pages/creator.py:51`), but the catalogue of named templates from the PRD is not confirmed present.

To do. Verify against the shipped Portal and fill in the missing patterns.

14.15 SDK C++ and Python surfaces

Covered as a defect in §16 because the documentation already claims these exist, but it belongs on the to-do list: achievements are reachable only from C today.

15. In development — the Achievement Platform initiative

An active delivery plan supersedes parts of §14. It lives at `../plans/achievement_matches/achievement-platform-epics-and-stories.md` and unifies three source plans into one sequenced initiative: the SDK alignment punch-list, a new derived-facts and trophy engine (`docs/achievement_engine_plan.md`), and new match ranking formats (`achievement-match-plan.md`). Each epic maps to a Linear project, each story to an SB-XXXX issue.

Anyone reading §14 should read this section first — several to-dos there are already designed and scheduled here.

15.1 Epics

Epic	Scope	Depends on
0 — SDK alignment punch-list	Closes out PR #2476: the <code>group_id</code> key fix, missing SDK tests, doc audit, dead JSON keys, backend tidy-ups	None
1 — Derived-facts substrate	<code>SessionTimelineRow</code> (raw CSV mirror in Postgres) + <code>SessionModeInstance</code> (one row per mode activation). Parse each session log from R2 exactly once; every downstream consumer then queries Postgres only	None
2 — Windowed + trophy machinery	Repeatable windowed unlocks and real holder-swap trophies with revoke semantics	Epic 1
3 — Match ranking foundation	<code>ranking_format</code> + <code>ranking_config</code> on <code>Tournament</code> , <code>RankingStrategy</code> interface, <code>HighScoreStrategy</code> wrapping today's behaviour with zero visible change	None
4 — First-to-Achieve matches	Match won by whoever triggers a tracked achievement first; hooks the existing unlock endpoint	Epic 3
5 — Counter matches + verification	Match ranked by a running counter, verified against the authoritative log after each session	Epics 1, 3
6 — Multi-machine + payout gating	Matches spanning several machines at a venue; withholds payout until every session is verified	Epics 4, 5

Recommended sequence: Wave A = Epics 0, 1, 3 in parallel · Wave B = 2 and 4 · Wave C = 5 · Wave D = 6. Match-facing behaviour ships behind a match-creator feature flag.

15.2 Contracts the initiative locks in

These are worth knowing even if you never touch match code, because they constrain the achievement paths documented above:

- `session_completed` remains the **sole** post-game trigger, for both fact derivation and match verification.
- `POST /api/v2/achievements/unlock/` remains the **sole** live-path ingest for unlocks, and it fans out to match event streams **before** the `UserAchievementV2.achieved` idempotency latch drops a repeat.
- `SessionTimelineRow` and `SessionModeInstance` become the canonical derived-facts tables. After Epic 1, nothing reads from R2 beyond initial ingest.
- A single metric-key namespace spans achievements and matches: `mode:<name>:duration_ms`, `mode:<name>:count`, `mode:<name>:score_gained`, `mode:<name>:max_score_per_activation`, `target:<key>:count`, `achievement:<key>:trigger_count`.

15.3 What this resolves from §14

§14 item	Resolved by	How
§14.2 TIMER / duration	Epic 1	<code>SessionModeInstance</code> stores one row per activation, so per-activation duration becomes a direct query; <code>mode:<name>:duration_ms</code> enters the metric vocabulary
§14.7 Relative targets	Epic 2.3	Trophy swap recomputes the top holder from <code>SessionModeInstance + Score</code> in a single SQL statement — the leaderboard predicate §14.7 says is missing
§14.8 Automatic trophy recomputation	Epic 2.2–2.4	New <code>AchievementHolder</code> table (<code>achievement</code> , <code>context_key</code> , <code>user</code> , <code>metric_value</code>), a <code>SELECT ... FOR UPDATE</code> swap transaction, and <code>update_trophy_holder</code> scheduled off the post-session chain
Trophy revoke notification	Epic 2.7	<code>achievement-trophy-revoked</code> template; every swap emits exactly two notifications — revoke to the prior holder, unlock to the new one

15.4 New concept — windowed achievements

Not described anywhere in the v3.0 specification and not in the current model, so flagged here:
Epic 2.1 makes an achievement unlockable **more than once**, scoped to a time window.

`UserAchievementV2` today permits exactly one unlock per `(user, achievement)` forever. Epic 2.1 adds:

- `context_key` — e.g. "2026-07" for a windowed unlock, `null` for a normal one. The unique constraint becomes `(user, achievement, context_key)`.
- `revoked_at` — set when a trophy transfers away from a holder.

Together these unblock "won June's award" and "won July's award" coexisting on one profile, and trophies that can genuinely be lost. Epic 2.5 freezes a window's winner into a permanent record (`context_key = "<window>:winner"`) so July's champion survives August's recompute.

The plan flags this as **the highest-risk migration in the project** — every consumer of `UserAchievementV2` must be enumerated before it merges.

15.5 Open questions

Called out in the plan as needing a decision before Epic 2.3–2.5 ship:

- Tie-breaking when two players have an identical `metric_value` — first-to-reach, alphabetical, random, or both hold.
- Non-trophy revocation policy on score deletion or admin reassignment. Current default is to grandfather the unlock.

16. Known defects

Tracked in the punch-list of
../plans/jsa-achievement-v2-sdk-alignment-plan.md .

#	Defect	Impact
1	groupid vs group_id — the SDK reads JSON key "groupid" (source/identifiers.h:148); the API emits "group_id" . nlohmann::json::value() returns the default silently, so Achievement.groupId is always 0 .	Groups and levels are inert on-device. Tracked as SB-4063 , PR scorbit_sdk#177.
2	No SDK achievement tests	Zero coverage of parsing, matching, or unlock handling.
3	Achievements are C-only	No public C++ methods, no Python bindings — while sdk-017 – 025 document all three, and the achievements.h header comment shows C++ calls that do not compile.
4	Session-limited progress never resets	clearUserProgress / clearAllProgress have no callers, so is_single_session is not enforced on-device.
5	ball_count mapped onto count	source/net.cpp:2198 reads the API's ball_count into the counter threshold. Needs a semantics decision — the API emits no count field.
6	level stored in achievementId	source/net.cpp:2206 puts the API's level in a field documented as "achievement ID within group." Functional but mislabelled.
7	Comparison operators can defeat the server fail-safe	< or = 0 on an SDK-evaluated rule type is trivially satisfied server-side (§5.3). No validation prevents authoring this.

17. Implementation status at a glance

Consolidated view of every capability in the v3.0 specification and both PRDs. This is the summary of what §4–§16 establish in detail — where the two disagree, the detailed section wins.

Legend: Implemented · Partial · Scheduled (see §15) · Not implemented · Implemented but defective

Taxonomy and properties

Capability	Status	Notes	Detail
Scope — game (title-level)	✓	The common case	§4.2
Scope — global	✓	Post-game evaluation only	§4.2
Scope — venue	⚠	Delivery is filtered by venue; unlock does not verify the player was there	§14.6
Scope — event	⚠	Same gap as venue	§14.6
Permanent achievements	✓		§4.3
Trophy — transfer mechanism	✓	Lock endpoint, events, notifications all exist	§7.5
Trophy — automatic recomputation	➡	Today a client must assert the transfer	§14.8
Windowed / repeatable unlocks	➡	Not in the v3.0 spec; new in Epic 2.1	§15.4
Session scope — server side	✓	is_single_session	§4.5
Session scope — SDK reset	✗	Limited progress never resets on device	§16
Visible / invisible (ingredient)	✓		§4.4
Obscured achievements	✓		§4.4
Draft / published / archived	✓		§11.3
Groups	🐛	Correct server-side; inert on device — group_id key mismatch	§16
Levels — sparse ordering + display_position	✓		§4.6
Nesting / sub-achievements	✓	Via ACHIEVEMENT rules	§4.7
Counters (boolean vs count)	✓		§4.8
Comparison operators, per rule	✓	> < = , author-selectable in the Creator	§4.9
Badge art, incl. GIF / Lottie	✓		§4.10
Obscure image	✓		§4.10
DMD frames — static, versioned ZIP	✓		§13.3
DMD frames — animated sequences	✗	Open question from the v2 PRD	§14.13

Rule types

Rule type	Status	Evaluated by	Detail
MODE	✓	SDK, in-session	§5.2
MODE_START	✓	SDK, in-session	§5.2
MODE_STACK	✓	SDK, in-session	§5.2
SCORE	✓	SDK, in-session	§5.2
PROGRESS	✓	Server, post-game	§5.2
ACHIEVEMENT	✓	Server, post-game	§5.2
GAME_CODE	✓	Game code, by design — never auto-unlocks	§14.1
TIMER	➡	Type and duration field exist; nothing evaluates them	§14.2
EVENT	✗	Type defined only	§14.3
ATTEMPT	✗	Type defined only	§14.4

Other rule ingredients from the specification

Ingredient	Status	Notes	Detail
Score thresholds (absolute)	✓		§5.2
Score targets (relative / "highest")	➡	Epic 2.3 covers the trophy path only, not general rules	§14.7
Duration	➡	Stored and editable; unevaluated	§14.2
Ball count	✗	Field stored, editable, migrated — no evaluator, no rule type	§14.5
Geo-fencing	✗	No location predicate exists	§14.12

Evaluation and processing

Capability	Status	Notes	Detail
In-session engine (SDK)	✓	Predictive only; game must ask	§6.1
Post-game engine (server)	✓	Authoritative	§6.1
session_completed trigger	✓	Sole post-game trigger	§6.4
Server fail-safe on SDK-owned rules	⚠	Defeated by < or = 0 ; nothing validates against it	§5.3
Explicit real-time vs post-game flag	✗	Routing is implicit from rules and scope; v1 had a flag	§14.11
Derived-facts substrate	➡	SessionTimelineRow + SessionModeInstance	§15.1
Ephemeral session unlock array (Redis)	✗	PRD requirement; never built	§14.10

Delivery and real-time

Capability	Status	Notes	Detail
<code>achievements:session:{uuid} channel</code>	✓		§9.1
Unlock / lock / progress events	✓		§9.2
Per-user / machine / game channels	✗	Three of four specified channels absent	§14.9
Push notifications to app + followers	✓		§13.2
Trophy revoke notification	→ soon	<code>achievement-trophy-revoked</code> template	§15.3
DMD frame display on unlock	✓	Frames pre-cached on device	§7.3

Authoring — Creator Portal

Capability	Status	Notes	Detail
Permission-gated access	✓	<code>creator</code> group + per-title/manufacture assignment	§11.1
Dashboard, list, search, filter	✓		§11.2
Creation wizard	✓	Boolean-algebra flow	§11.2
Per-rule operator selection	✓	Contextual labels	§4.9
Achievement templates	⚠	Loader exists; PRD's named catalogue unconfirmed	§14.14
DMD pixel-art editor	✓	Palette, fonts, layers, transparency	§11.4
Draft → publish → archive	✓		§11.3
Testing / trigger to own account	✓		§11.5
CSV import	⚠	Transitional; deprecated in favour of the Portal	§13.4

SDK

Capability	Status	Notes	Detail
C API	✓	18 functions, complete	§10.2
C++ API	✗	Types only — no methods on public <code>GameState</code>	§10.1
Python bindings	✗	None exist	§10.1
Definition + progress caching	✓		§10.3
Local rule matching	✓	Must be invoked by game code	§10.3
DMD frame LRU cache	✓	32 frames	§3.3
Achievement unit tests	✗	Zero coverage	§16

18. Glossary

Term	Meaning
Achievement	A badge attached to a user profile for a playing accomplishment
Trophy	An achievement only one player can hold; transfers when someone else qualifies
Lock / unlock	Unlock = grant. Lock = revoke, used for trophy transfer.
Rule	One condition; an achievement's rules are ANDed
Sub-achievement	A prerequisite achievement referenced by an ACHIEVEMENT rule
Invisible	Never shown to players; an ingredient in a chain
Obscured	Shown but masked until earned
Group / level	Association and ordering for a progression ladder
Mode	A named boolean game state — the substrate for most rules
Session	One game from start to upload; the unit of real-time scoping
DMD frame	Pixel art sized to a specific machine's dot-matrix display
Scorbitron	Scorbit hardware that infers game state on machines without native integration
In-session / post-game	The two evaluation engines (§6)

Maintained in [eng_docs/reference/](#). Related: [achievement-key-format.md](#) · [centrifugo-ops-guide.md](#) · [architecture.md](#) · [../plans/jsa-achievement-v2-sdk-alignment-plan.md](#).